Low-Level Design (LLD)

RAG-Based Customer Support Assistant with LangGraph & HITL

RAG Customer Support Assistant | v1.0 | Innomatics Research Labs

1. Module-Level Design

ingest.py

Role	Document Processing, Chunking, Embedding, Vector Storage
Description	Loads PDF via PyPDFLoader. Splits using RecursiveCharacterTextSplitter

retriever.py

Role	Retrieval Module
Description	Loads ChromaDB from disk. Initialises retriever with top-k=3 similarity search

llm.py

Role	Query Processing / LLM Module
Description	Initialises Groq ChatGroq with llama-3.1-8b-instant. Applies PromptTemplate

graph.py

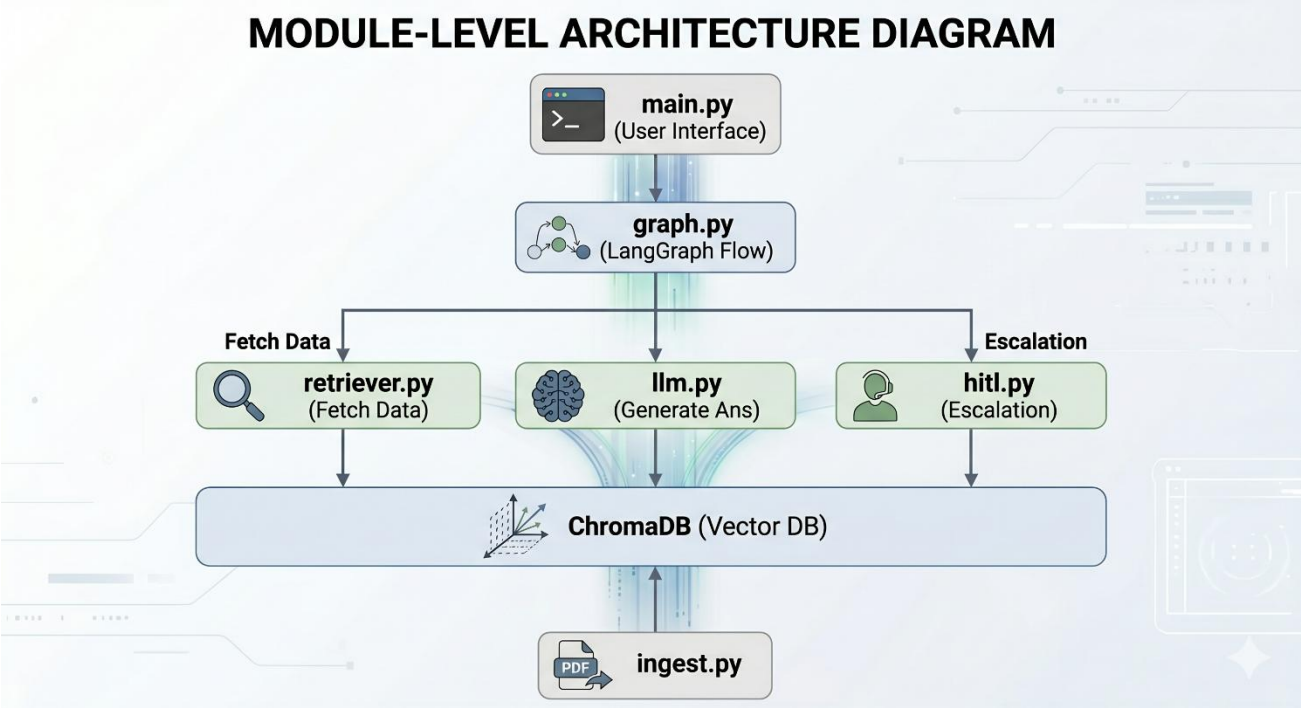
Role	Graph Execution Module
Description	Defines GraphState TypedDict. Builds LangGraph StateGraph with process_

hitl.py

Role	HITL Module
Description	Detects escalation from graph state. Prints alert to terminal. Accepts input

main.py

Role	Interface / Entry Point
Description	Displays welcome banner. Runs interactive while loop.


MODULE-LEVEL ARCHITECTURE DIAGRAM
main.py
(User Interface)
graph.py
(LangGraph Flow)
Fetch Data
Escalation
retriever.py
(Fetch Data)
llm.py
(Generate Ans)
hitl.py
(Escalation)
ChromaDB (Vector DB)
PDF
ingest.py

2. Data Structures

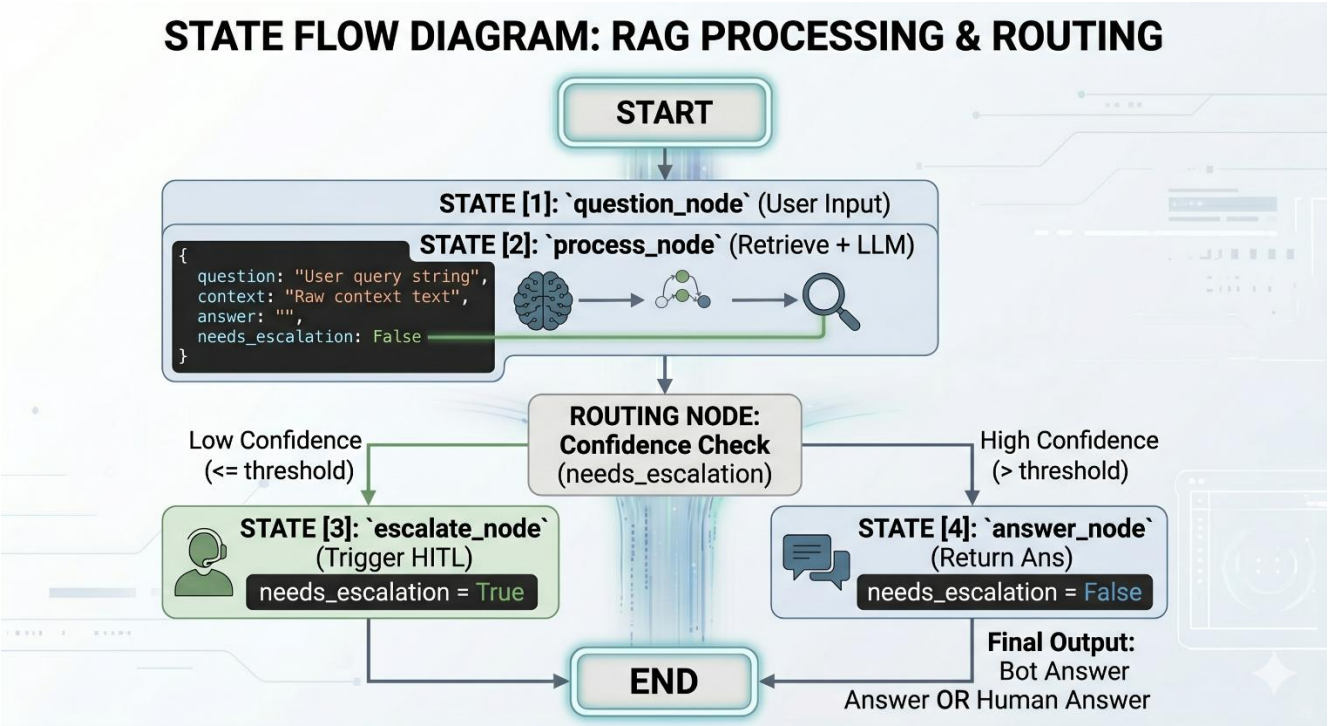
GraphState (TypedDict)

The central state object that flows between all LangGraph nodes:

```
class GraphState(TypedDict):
question: str # Original user question
context: str # Retrieved chunks (formatted string)
answer: str # LLM-generated answer
needs_escalation: bool # True if HITL is required
```

Document / Chunk Format

Field	Type	Description
page_content	str	The raw text of the chunk
metadata.source	str	File path of the source PDF
metadata.page	int	Page number within the PDF



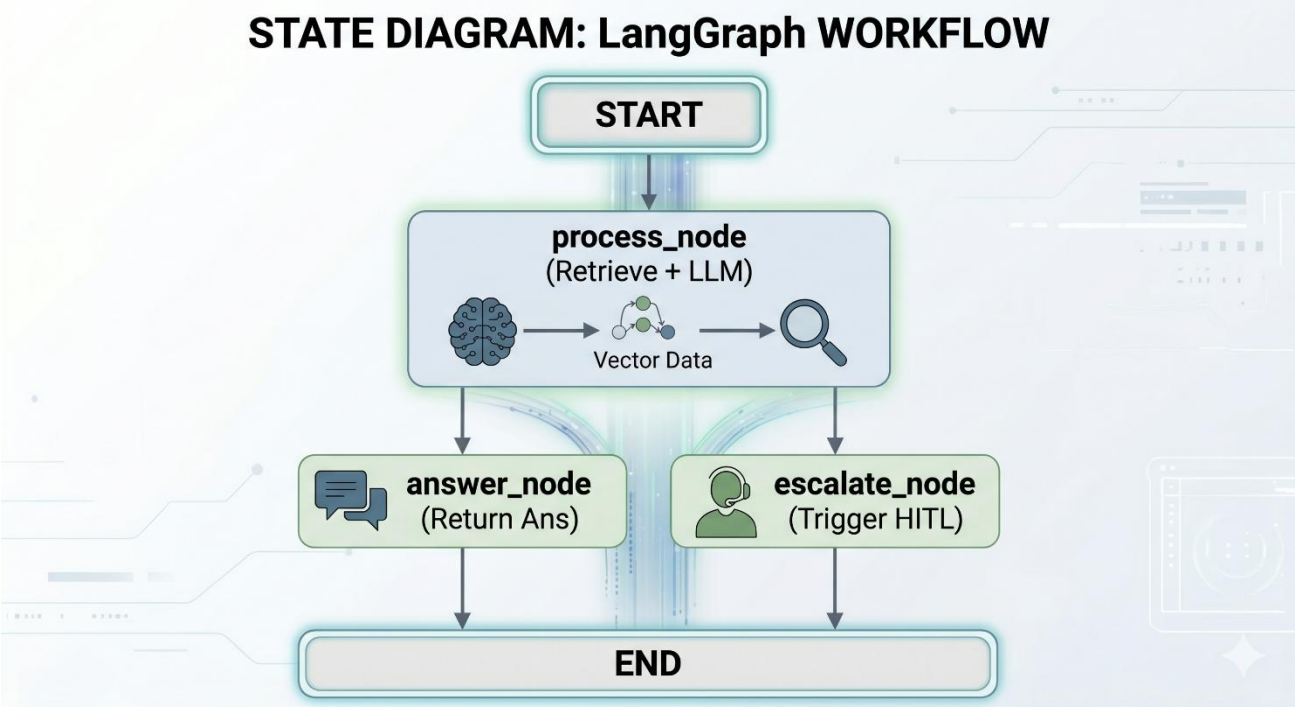
3. Workflow Design (LangGraph)

Nodes

Node Name	Function	Input State Keys	Output State Keys
process	process_node()	question	context, answer
answer_node	answer_node()	answer	needs_escalation=False
escalate_node	escalate_node()	answer	needs_escalation=True

Edges & Routing

- Entry point: process node
- Conditional edge from process: calls route_query(state) which returns 'answer' or 'escalate'
- answer route: process -> answer_node -> END
- escalate route: process -> escalate_node -> END



4. Conditional Routing Logic

The router scans the LLM-generated answer (lowercased) for the following phrases:

- i don't have enough information
- let me escalate
- cannot answer
- not in the context
- i'm not sure
- unclear

If any phrase is found → route to escalate_node. Otherwise → route to answer_node. This approach is lightweight and deterministic, avoiding an extra LLM call for confidence scoring.

5. HITL Design

Aspect	Detail
Trigger condition	needs_escalation == True in GraphState after routing
Escalation alert	Prints a formatted alert block with question and bot answer to terminal
Human input	input() prompt receives free-text response from the agent
Fallback	If agent types nothing, a default message is returned
Logging	All escalations logged to escalation_log.txt with timestamp
Return	Human response is returned as final answer to the user

6. Interface Design

Input Format

- Free-text string typed by user in CLI
- Special commands: 'quit', 'exit', 'q' to terminate

Output Format

- Bot answer: prefixed with '■ Bot:'
- Human agent answer: prefixed with '■ Agent:'
- Escalation alert: formatted block with question + bot answer

7. Error Handling

Error Scenario	Handling Strategy
PDF file not found	PyPDFLoader raises FileNotFoundError — user must ensure knowledge_bas e.pdf exists in da
No relevant chunks found	Retriever returns empty list — format_context returns empty string — LLM wi ll trigger escalatio
LLM API failure (Groq)	LangChain raises exception — wrap generate_answer in try/except to return fallback escalatio
Empty user input	main.py checks for empty string and skips with continue
Human agent types nothing	hitl.py returns default message: 'A human agent has been notified...'